

경력기술서

가능성을 기회로 바꾸고, 제품으로 가치를 전하는
메이커 김대정입니다.

김대정

Product Engineer · Tech Lead

marin.backend@gmail.com

github.com/Dae-Jeong

marinkim.xyz

경력 요약

제품·Backend·AI

기획자로 시작해 백엔드로 왔고, 아이디어를 제안한 AI 콘텐츠 제품의 FastAPI 백엔드와 AI 생성·평가 시스템을 직접 만들고 핵심 화면은 coding agent로 완성해 월 1천만원 수준의 구독 매출이 발생하는 제품으로 운영합니다. AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀에서, 백엔드 경험이 적은 팀원도 같은 기준으로 만들 수 있게 FastAPI 조직 표준과 agent 작업 맥락, evidence로 담은 QA 판정 규칙을 직접 세웠습니다.

이 문서는 이력서의 성과를 같은 순서로 풀어 각 항목에서 무엇이 문제였고 무엇을 판단했으며 어떤 경계를 구현하고 어떻게 검증했는지를 적었습니다. 화면은 coding agent로 구현하고 직접 검수했으며 수치는 측정된 것만 썼습니다.

MediSolve AI

Tech Lead · Backend Engineer

2025.04 - 현재 · 더데이랩스
프리랜서 2025.02 - 2025.04

AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀에서, 제품별 결정·QA·릴리스 운영을 리드하고 Backend·AI application을 직접 구현했습니다. 팀이 같은 기준으로 만들 수 있는 Backend Template과 QA 판정 규칙은 직접 설계했습니다.

조직 표준 · 팀이 같은 기준으로 만드는 FastAPI template과 evidence로 닫는 QA 판정 규칙

팀 공통 Backend 기준과 QA 판정 규칙

담당 범위	Template 설계·구축 · QA 판정 규칙 설계 · 도입 지원
문제	팀이 coding agent로 기능을 만들기 시작하면서 백엔드 경험이 적은 담당자도 구현에 참여했습니다. QA와 운영 준비 단계에서 사용량이나 동시 요청 조건에 따라 구조적인 문제가 자주 드러나 백엔드 엔지니어의 리소스가 원인 파악과 보완에 쓰였고, QA는 실행이 성공하면 통과로 보는 경우가 있어 AI 기능의 품질 판정이 흐려졌습니다.
선택	기능마다 완벽한 구조를 요구하는 대신 자주 틀리는 경계를 기본값으로 제공했습니다. Service가 transaction 정책을 선언하고 Repository는 현재 session만 resolve하게 했고, tool이 늘어나는 Agent 기능만 Hexagonal, 나머지는 MVC를 기본 구조로 두었습니다. QA는 요구사항을 REQ로 쪼개 evidence로 닫는 판정 규칙을 세워 실행 성공과 품질 통과를 분리했습니다.
구현	• @transactional(REQUIRED·REQUIRES_NEW·NESTED)·ContextVar 기반 AsyncSession resolve-owner-task guard, typed DTO 경계, naming·timezone·soft delete·cursor pagination convention, ADR·runbook·agent 작업 맥락을 template에 넣었습니다. • 전 제품 공통 Quality Evidence Harness를 QA 팀원의 서포트를 받아 작성했습니다. REQ별 evidence plan(FE·DB·Log·Network·AI quality), PASS/FAIL/UNKNOWN 판정, 반복 이슈의 자동화 승격, LLM judge 단독 승인 금지가 규칙이며, QA 팀이 운영하는 AI QA 에이전트 파이프라인의 판정 layer로 연결했습니다.
검증	• propagation·isolation·read-only·CancelledError rollback·connection cleanup을 integration test로 고정했고, 정기 개발 회의에서 패턴 선택 기준과 구조화 비용을 검토해 반영했습니다.

결과	full template으로 시작한 신규 프로그램의 STG QA에서 같은 유형의 session-connection 문제가 재관측되지 않았고, 기획·QA·디자인 담당자가 직접 구현하는 동안 백엔드는 결과 피드백과 배포 지원으로 개입을 줄였습니다. 범위 · Template과 판정 규칙의 설계는 직접 했고 각 기능 구현과 QA 에이전트 플러그인 구현은 담당자의 몫입니다. 개발 속도·결함 수의 수치는 측정하지 않았습니다.
----	---

Thready Backend 재구축 · 규칙을 먼저 설계하고 돌아가는 제품을 멈추지 않고 교체

기존 frontend contract를 유지한 병렬 재구축과 단계적 전환, 이후 실사용 backend 운영 전담

담당 범위	대안 비교·설득·설계·구현·검증·전환 판단 직접 수행
문제	빠른 검증 중심으로 만들어진 초기 backend는 도메인 의존성이 얹혀 회원 로직 변경이 AI 생성 중단으로 이어졌고 해결된 QA 이슈가 같은 영역에서 다른 형태로 재발했습니다.
선택	부분 수정을 누적하는 안과 backend만 병렬 재구축하는 안을 비교해 서비스가 작고 AI 모듈 확장이 예정된 시점이라 재구축을 택하되 기존 frontend와 릴리스 흐름은 유지하는 범위로 한정했습니다. "돌아가는 기능을 왜 다시 만드나"에는 문제 누적 속도·AI 확장성·하네스 기반 이관 속도로 답했습니다.
구현	- 패턴·계층·검증 하네스를 먼저 세우고 그 규칙 위에서 API·기능 inventory를 만든 뒤 새 FastAPI backend를 나란히 구현했습니다. - domain·repository·transaction 책임을 분리하고 전환 단위를 release로 관리해 frontend 호출을 단계적으로 옮겼습니다. - 재구축 범위·architecture·검증·전환 판단은 직접 소유하고 coding agent는 codebase 파악·기능 inventory·반복 구현에 썼습니다.
검증	동일 기능의 응답 비교와 QA acceptance를 통과한 범위만 전환했고 전환 뒤에도 같은 Jira 정의로 재발을 계속 측정했습니다.

결과	같은 기준의 Jira 집계에서 해결된 QA 이슈 재오픈 비율이 37%에서 11%로, 재발 발생 일평균이 약 94% 줄었습니다(하루 4.5건에서 0.3건). 전환 이후 실제 사용자가 쓰는 backend의 배포·QA·운영을 계속 전담하고 있습니다. 범위 · 티켓에 BE/FE 라벨이 없어 제품 전체 품질 지표로 서술합니다. 초기 prototype은 다른 engineer가 만들었습니다.
----	---

대화형 제품 제어 Agent · 계획과 실행 권한을 나누고 변경은 사람 확인과 먹등 receipt를 거치는 prototype Thready 기능을 대화로 제어하는 흐름의 성립 여부와 변경 작업의 위임 범위 검증	
담당 범위	planner-executor-capability registry·상태 원장 설계·구현·검증
문제	원하는 작업을 말하면 제품이 허용된 기능을 골라 실행하는 흐름이 실제로 성립하는지, 예약·발행·삭제 같은 변경 작업을 어디까지 agent에 맡길지 검증이 필요했습니다.
선택	multi-agent 대신 planner-executor 하나로 두고 planner는 typed plan만 만들게 했습니다. 실행은 capability registry가 등록·일치를 확인한 action만 dispatch하고 변경 작업은 다음 turn의 typed confirmation과 receipt idempotency를 통과할 때만 상태를 바꿉니다.
구현	• 글감 탐색·기획·작성·수정 7개 편집 action과 계정·게시물·성과 13개 운영 action을 capability로 등록하고 편집은 기존 writer를 재사용했습니다. • conversation·message·turn·tool result·versioned artifact를 분리한 원장과 append-only activity event, token-aware context selection과 compaction snapshot을 구현했습니다. • 명확한 읽기 요청은 deterministic fast lane으로 처리하고 모호한 mutation 요청은 planner fallback으로 실행하지 않게 했습니다.
검증	• 승인 전 무변경·승인 뒤 상태 전이·새로고침 뒤 activity와 artifact 복원을 확인했고 prototype 전체 test 679개가 통과했습니다.
결과	agent가 준비할 일과 사람이 확정할 일의 경계를 코드로 검증했습니다. 범위 · 독립 prototype입니다. 운영 action은 Mock gateway까지만 연결했고 실제 예약·발행·삭제와 운영용 durable worker는 구현 범위 밖입니다.

Thready 생성 파이프라인 · LangGraph 그래프를 단일 파이프라인으로 축소하고 SDK agent 엔진과 실측 비교

Thready AI 생성·역할·소재 추출 그래프의 구현·운영과 legacy 그래프 정리, 생성 엔진 두 개의 병존

담당 범위	생성 그래프 설계·구현·운영 · legacy 축소와 엔진 비교 실측 직접 수행
문제	생성 그래프가 여러 노드와 조립기·guard 엔진으로 불어나 기능 하나를 바꿔도 손덜 곳이 많아졌고, 한 번 생성하는 데 걸리는 시간도 함께 길어졌습니다. 유형 분기 판정을 writer에 두었을 때는 판정이 발동하지 않는 문제도 있었습니다.
선택	15노드 legacy 그래프를 publishable 단일 파이프라인으로 축소하되 노드명은 trace 연속성을 위해 유지했습니다. 그래프를 걷어내는 대신 SDK 기반 단일 agent 엔진을 같은 guard·repair 코드로 병존시켜 실측으로 비교했고, 판정은 writer가 아니라 planner가 내리도록 위치를 재배치했습니다.
구현	• LangGraph StateGraph로 생성 파이프라인과 역할·소재 추출 그래프를 구현하고, LLM provider·tool처럼 치환이 실재하는 경계에만 port를 두고 그 위에 그래프 application을 올렸습니다. • 그래프 컴파일은 첫 생성까지 미루는 lazy 구성으로 두고, legacy 그래프·조립기·guard 형식 엔진을 걷어내 publishable 그래프 하나로 축소했습니다. • SDK 기반 단일 agent 엔진을 그래프 엔진과 같은 guard·repair 코드를 재사용하도록 두고 설정 분기로 두 엔진을 병존시켰습니다.
검증	• 실 파이프라인 E2E에서 agent 엔진 43초 vs 그래프 294초로 실행 시간을 비교했고, 축소 뒤 전체 테스트와 guard를 통과했습니다.

결과	생성 그래프가 판단 위치가 분명한 단일 파이프라인으로 정리됐고, 실행 시간 비교 근거를 가진 두 엔진을 같은 guard 위에서 운영할 수 있게 됐습니다. 범위 · 멀티에이전트 오케스트레이션 플랫폼 운영이 아닙니다. 43초/294초는 엔진 비교 실측이며 품질 수치가 아닙니다. 내부 규모 수치는 공개하지 않습니다.
----	--

Thready · 반복하던 글 제작을 제품 흐름으로, 글의 기준은 실측으로 신뢰를 쌓은 유료 AI 제품

아이디어 제안 → prototype 이후 제품화 주도 → 고객이 구독하는 서비스 운영

담당 범위	제품 운영 리드 · Backend/AI 직접 구현 · 핵심 화면은 coding agent로 완성
문제	여러 채널을 운영하는 고객은 자료 수집·작성·검수·예약을 매번 반복했고 반응이 좋은 글의 기준은 감에 의존했습니다.
선택	글쓰기만 AI에 맡기지 않고 자료 수집·정체성·작성·검수 역할을 나눠 AI는 초안과 1차 검수를, 사람은 최종 수정과 발행 판단을 맡게 했습니다. 좋은 글의 기준은 판정기가 아니라 판단 데이터를 쌓는 장치로 LLM judge를 두고 기준 자체는 직접 수집한 실측 corpus로 다시 썼습니다.
구현	• FastAPI 제품 API와 별도 AI application·DB를 인증된 HTTP 경계로 분리했습니다. 콘텐츠 가져오기·URL preview·source 검증과 생성·예약·발행·대시보드·관리 화면은 coding agent로 구현해 직접 검수·배포했습니다. • typed prompt builder·planner/writer 생성·LLM judge·사람 라벨링을 분리해 평가 이력을 남기고 대량 corpus importer는 (source, source_key) upsert와 batch rollback으로 재적재를 먹등하게 만들었습니다. • 품질 기준값이 자사 출력을 되먹이던 순환을 재실측으로 발견해 교정하고 반증된 접근은 '다시 시도하지 말 것' 로그로 남겨 실험 범위를 좁혔습니다.
검증	• local 격리 DB에서 corpus를 두 번 적재해 건수 불변과 기존 라벨 보존을 확인했고 release·QA·task 기준으로 실제 사용자 흐름의 회귀를 확인했습니다. • 외부 AI 5xx는 실패 유형으로 분류해 재시도·최종 실패·사용자 재시도 경로를 두고 반복 장애 모델은 Sentry 확인 뒤 선택지에서 일시 제외했습니다.

결과	기획·QA·마케팅과 제품 운영을 리드해 실제 고객이 구독료를 내는 제품으로 출시·운영 중입니다. 범위 · 매출은 팀·제품의 결과입니다. 초기 prototype은 다른 engineer가 만들었고, 제 범위는 아이디어 제안과 prototype 이후 제품화·Backend·AI·운영 리드입니다.
----	--

주문 후속 작업과 실시간 상담 · 실패를 상태로 승격하고 순서를 보장

피부와 CRM의 주문·재고 worker(주도)와 실시간 AI 상담 runtime(공동 주 기여)

담당 범위	주문·재고 backend·worker 흐름 주도 · 실시간 상담 session·provider 경계 공동 안정화
문제	주문 생성 뒤 공급사·병원 알림이 외부 연동에서 따로 실패할 수 있었고, 실시간 상담에서는 외부 음성 모델 세션이 발화 중간에 끊기거나 보정 결과가 늦게 도착해 문맥이 바뀔 수 있었습니다.
선택	실패 가능한 작업은 API 요청 밖 worker 경계에서 처리하고 실패를 숨기지 않고 주문 상태로 승격했습니다. 실시간 상담은 세션이 끊겨도 전사가 멈추지 않도록 겹치는 세션 폴로 흡수하고, 순서는 도착 순서가 아니라 sequence가 정하게 했습니다.
구현	• 기존 task queue가 asyncio 실행 모델과 맞지 않아 교체하면서 알림 job의 PENDING → SENDING → SUCCESS/FAILED 상태, 고정 간격 재시도, 최종 실패 이력, 전용 API의 수동 재발송을 주문 상태와 연결했습니다. • 외부 음성 모델의 세션 종료 문제를 겹치는 window로 여러 세션을 시간차로 열고 세션별 버퍼와 sequence로 병합해 흡수했고, 비용 증가와 첫 응답 지연은 문서로 명시한 뒤 모델 교체 후 우회 코드를 걷어냈습니다. • DELTA·COMPLETE·CORRECTED를 같은 sequence로 묶어 늦게 도착한 보정이 다른 turn을 덮지 않게 했고, 종료 뒤 재연결되던 경합 경로의 정리 책임과 stop guard를 보강했습니다.
검증	• VAD 파라미터별 E2E 실측에서 P50 차이가 작고 주 병목이 모델 추론임을 실측해, VAD tuning 대신 DELTA 조기 trigger에 집중했습니다. • 13개 focused regression으로 회귀를 고정했고, 상담 길이 E2E에서 sequence 누락·중복 없음을 확인했습니다.

결과	API 성공 뒤 실패를 숨기지 않고 운영자가 복구 지점을 확인할 수 있는 흐름과, 외부 모델이 끊겨도 순서가 유지되는 실시간 전사 경계를 확보했습니다. 범위 · 주문 worker는 주도, 실시간 상담은 공동 주 기여입니다. 전환에 따른 성능·지연 개선 수치는 측정하지 않았습니다.
----	--

제품 개발 체계 · 판단의 정확이 유지되는 기록 구조와 evidence로 닫는 QA 판정 규칙

제품별 결정·명세·작업·QA·릴리스 운영 리드, QA 팀과의 하네스화, 회사 AX 구조 설계 참여

담당 범위	제품 단위 운영 리드 · QA 판정 규칙 설계 · 회사 AX 설계 참여
문제	제품 간 맥락 공유가 사람에 의존해 요구·판단을 재확인하는 비용이 컸고 담당 교체·병행 시 실행이 지연되는 병목이 있었습니다. QA는 실행이 성공하면 통과로 보는 경우가 있어 AI 기능의 품질 판정이 흐려졌습니다.
선택	결정·명세·작업·릴리스 근거를 실행 원장으로 축적해 사람과 AI가 같은 맥락을 읽게 하되, 우선순위·승인·릴리스처럼 판단이 필요한 단계는 사람이 확정하는 경계를 지켰습니다. QA는 요구사항을 REQ로 쪼개 evidence로 닫는 판정 규칙을 세우고 실행 성공과 품질 통과를 분리했습니다.
구현	- 제품별 pipeline registry에 결정·명세·작업 기록을 실행 원장으로 적용하고 BE·FE·QA 담당 lane과 QA 승인을 release gate에 연결해 release note 생성을 자동화했습니다. - 전 제품 공통 Quality Evidence Harness를 QA 팀원의 서포트를 받아 작성했습니다. REQ별 evidence plan(FE·DB·Log·Network·AI quality), PASS/FAIL/UNKNOWN 판정, 반복 이슈의 자동화 승격, LLM judge 단독 승인 금지가 규칙입니다. - 이 판정 layer를 QA 팀이 운영하는 AI QA 에이전트 파이프라인(시나리오→TC→자동화→버그 등록)에 연결하고 백그라운드 작업의 고장 모드 카탈로그와 fault-injection QA를 설계로 제안했습니다.
검증	팀 주 1회 agent 활용 회고로 무엇을 만들지와 구현 방법을 함께 정했고 Slack에서 접수한 이슈를 사내 시스템에 남겨 해결 상태를 추적하는 흐름을 제안했습니다.

결과	담당자가 바뀌어도 기록에서 업무를 이어가 인수인계·맥락 복원 비용이 낮아졌고 유사 기능에서 기존 판단을 재사용해 lead time 단축에 기여했습니다. 이 구조를 회의·의사결정·업무 배정·승인으로 넓히는 회사 AX 설계에 참여했습니다. 범위 · 전사 문서 규칙 정의와 QA AI 에이전트 플러그인 구현은 다른 담당자가 주도했습니다. 제 범위는 제품 단위 적용·운영 리드, 공통 QA 판정 규칙 설계, AI QA 팀 운영 참여입니다. lead time·QA 시간의 수치는 측정하지 않았습니다.
----	--

Thready AI 실행부 · 전달 보장·먹등 소비·정합 검증으로 원장과 AI 상태를 분리

AI 실행부의 독립 application·DB 분리, STG 실테이터 이관, 서비스 간 durable delivery, 생성 원장의 경합 중재

담당 범위	경계 설계·구현·migration·검증 직접 수행
문제	제품 정책·원장과 AI 생성 lifecycle이 한 backend·DB에 있어 AI 확장과 장애가 원장에 결합됐고 분리하면 전달 유실과 역순 도착이 최신 상태를 덮을 수 있었습니다. 다중 worker가 같은 생성 원장을 두고 경합했습니다.
선택	DB를 공유하지 않고 독립 FastAPI application·DB로 분리해 인증된 HTTP 계약으로만 연결하고 전달은 원장 변경과 같은 transaction에 기록하는 Transactional Outbox로 두었습니다. 생성 원장은 상태 전이 규칙을 entity에 복원하고 version CAS로 경합을 중재했습니다.
구현	• relay는 짧은 lease로 row를 claim하고 delivery version·attempt count를 fencing token으로 붙이며, 소비 쪽은 더 높은 version만 반영하고 stable id·natural key 충돌을 최신 row 하나로 수렴시키는 먹등 upsert/delete를 소유합니다. 최대 시도를 넘긴 전달은 terminal failure로 보존합니다. • 생성 원장은 optimistic lock으로 worker 승자를 정하고 먹등 replay 판별과 전이가 같은 판정을 쓰게 했습니다. quota는 예약 시점에 잡고 admission gate가 확정 사용량과 진행 중 예약을 함께 봐 동시 요청의 초과 실행을 막습니다. • STG 생성 이력·품질 기록·실행 추적을 parent→child 순서로 streaming copy했고 영구 cross-DB link는 쓰지 않았습니다.
검증	• local rehearsal·건수·id·status·사유를 결합한 MD5 fingerprint·FK orphan 0건으로 이관 정합성을 확인했고 stale PUT/DELETE fence test와 양쪽 서비스 전체 회귀를 통과했습니다. • health가 성공해도 생성이 실패한 사례를 계기로 배포 성공과 기능 동작을 분리한 post-deploy 생성 API E2E gate를 세웠습니다.
결과	독립 AI application·DB를 STG·Prod에서 운영 중이며 지연·중복·역순 전달이 최신 원장 상태를 덮지 않는 복구 경계를 확보했습니다. 범위 · Prod migration 완료와 무중단 전환은 주장하지 않습니다.

Memento AI

2024.10 - 2025.01

Backend Engineer

예약·결제·고객 알림 Backend를 구현했습니다. 회사 폐업으로 경력이 종료되었습니다.

예약·결제·고객 알림 Backend

FastAPI·SQLAlchemy·MySQL 기반 예약 서비스

담당 범위	예약 API·외부 결제·알림 작업 구현
문제	외부 결제 결과와 내부 예약 자산 변경이 한 번에 성공하지 않을 수 있었습니다.
선택	결제 취소·환불과 내부 상태 변경 시점을 분리해 실패 지점을 확인할 수 있게 했습니다.
구현	- 예약 API와 외부 결제를 연동하고 결제 상태에 따라 내부 자산 변경을 제어했습니다. - 다국어 알림의 즉시·예약 발송과 작업 취소·재등록·발송 이력을 구현했습니다.
검증	결제·예약·알림 상태를 분리해 실패 시 재처리 범위를 확인했습니다.
결과	예약과 결제, 후속 알림의 책임을 나눠 운영 가능한 상태 흐름을 만들었습니다.

STUDIO LAB

2021.12 - 2024.01

Vision AI Engineer → Product Manager → Backend Engineer

AI 모델에서 시작해 제품 기획과 Backend까지 역할을 넓히며 생성형 AI 커머스 제품을 만들었습니다.

SellerCanvas · 생성형 AI 커머스 제품

prototype에서 v1.0과 외부 기업 PoC까지 발전

담당 범위	PM 중심 · AI/Backend 경험을 바탕으로 제품 범위와 출시 우선순위 결정
문제	AI 기능을 데모가 아니라 실제 업무 흐름에서 쓸 수 있는 제품으로 구체화해야 했습니다.
선택	사용자 흐름과 기능 범위를 먼저 정하고 모델·Backend·화면을 제품 단위로 연결했습니다.
구현	프로토타입의 기능 범위를 정리하고 v1.0 제품과 기업 PoC로 확장했습니다.
검증	사용 시나리오와 PoC 요구를 기준으로 기능 범위와 우선순위를 조정했습니다.

결과 CES 2024 Best of Innovation AI 부문 대상 제품에 참여했습니다.

범위 · 회사·팀 제품 성과이며 개인 단독 수상으로 표현하지 않습니다.

TellingMe · 개인 프로젝트

2024.01 - 2024.12

Backend Lead · Infra

Memento AI 재직 기간과 일부 겹치는 개인 프로젝트이며 정규 경력과 분리해 기록합니다.

TellingMe · 개인 프로젝트 Backend

Spring Boot Backend와 AWS 배포

담당 범위	Backend Lead · Infra
문제	개인 프로젝트의 API와 배포 환경을 한 흐름으로 운영해야 했습니다.
선택	Spring Boot Backend와 AWS 기반 배포 구성을 함께 맡았습니다.
구현	Backend API 개발과 서비스 배포 환경을 구성했습니다.
검증	개인 프로젝트의 운영 범위에서 배포와 기본 운영을 확인했습니다.

결과 제품 Backend와 배포를 함께 맡은 개인 프로젝트 경험을 확보했습니다.

범위 · 정규 경력이 아니며 개인 프로젝트로만 표기합니다.

아이즈슬

2020.08 - 2021.06

Vision AI Engineer · Intern

Vision AI 모델과 데이터 파이프라인을 제품 기능으로 연결하는 개발·검증에
참여했습니다.

기술

Backend

Python · FastAPI · PostgreSQL · MySQL · Redis · SQLAlchemy

Async / Realtime

RabbitMQ · TaskIQ · WebSocket · SSE · STT

AI Product

LLM integration · typed prompt · structured output · evaluation · labeling

Delivery

Docker · GitHub Actions · Terraform · Azure · AWS · Kubernetes(kubeadm · Calico
· MetalLB · ArgoCD)
